

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII ȘTIINȚIFICE



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

FPWAM - O implementare a mașinii abstracte Warren pe FPGA

LUCRARE DE LICENȚĂ

Absolvent: **Bogdan Daniel ARDELEAN**
Conducător științific: **Dr. Ing. Zoltan Francisc BARUCH**

2017



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

DECAN,
Prof. dr. ing. Liviu MICLEA

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Bogdan Daniel ARDELEAN**

FPWAM - O implementare a mașinii abstracte Warren pe FPGA

1. **Enunțul temei:** *Scurtă descriere a temei lucrării de licență și datele inițiale*
2. **Conținutul lucrării:** *(enumerarea părților componente) Exemplu: Pagina de prezentare, aprecierile coordonatorului de lucrare, titlul capitolului 1, titlul capitolului 2, titlul capitolului n, bibliografie, anexe.*
3. **Locul documentării:** *Exemplu: Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare*
4. **Consultanți:**
5. **Data emiterii temei:** 1 Noiembrie 2015
6. **Date predării:** 17 Februarie 2017 *(se va completa data predării)*

Absolvent: _____

Coordonator științific: _____



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

**Declarație pe proprie răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a)

_____, legitimat(ă)
cu _____ seria _____ nr. _____
CNP _____, autorul lucrării _____

elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*.

Data

Nume, Prenume

Semnătura

De citit înainte (această pagină se va elimina din versiunea finală):

1. Cele trei pagini anterioare (foaie de capăt, foaie sumar, declarație) se vor lista pe foi separate (nu față-verso), fiind incluse în lucrarea listată. Foaia de sumar (a doua) necesită semnătura absolventului, respectiv a coordonatorului. Pe declarație se trece data când se predă lucrarea la secretarii de comisie.
2. Pe foaia de capăt, se va trece corect titulatura cadrului didactic îndrumător, în engleză (consultați pagina de unde ați descărcat acest document pentru lista cadrelor didactice cu titulaturile lor).
3. Documentul curent **nu** a fost creat în MS Office. E posibil să fie mici diferențe de formatare.
4. Cuprinsul începe pe pagina nouă, impară (dacă se face listare față-verso), prima pagină din capitolul Introducere tot așa, fiind numerotată cu 1.
5. Vizualizați (recomandabil și în timpul editării) acest document
6. Fiecare capitol începe pe pagină nouă.
7. Folosiți stilurile predefinite (Headings, Figure, Table, Normal, etc.)
8. Marginile la pagini nu se modifică.
9. Respectați restul instrucțiunilor din fiecare capitol.

Cuprins

Capitolul 1	Introducere - Contextul proiectului	11
1.1	Contextul proiectului	11
1.1.1	FPGA	11
1.1.2	Prolog	12
1.1.3	Nu m-am decis	12
Capitolul 2	Obiectivele Proiectului	13
2.1	Scop	13
2.2	Obiective generale	13
2.3	Obiective S.M.A.R.T.	14
Capitolul 3	Studiu Bibliografic	17
3.1	Sisteme de calcul reconfigurabile și FPGA	17
3.1.1	Circuitele integrate FPGA	17
3.2	Prolog și arhitectura WAM	18
Capitolul 4	Analiză și Fundamentare Teoretică	20
4.1	Titlu	20
4.2	Alt titlu	20
Capitolul 5	Proiectare de Detaliu și Implementare	21
Capitolul 6	Testare și Validare	22
6.1	Titlu	22
6.2	Alt titlu	22
Capitolul 7	Manual de Instalare și Utilizare	23
7.1	Titlu	23
7.2	Alt titlu	23
Capitolul 8	Concluzii	24
8.1	Titlu	24
8.2	Alt titlu	24

Bibliografie	25
Anexa A Secțiuni relevante din cod	26
Anexa B Alte informații relevante (demonstrații etc.)	27
Anexa C Lucrări publicate (dacă există)	28

Capitolul 1

Introducere - Contextul proiectului

1.1 Contextul proiectului

Încă din secolul XVII, cu mult înaintea calculatoarelor electronice, exista termenul *calculator*. Până la jumătatea secolului XX, denotația lui era cu totul alta față de cea pe care o știm noi astăzi. Sensul acestui cuvânt referindu-se în principal la o meserie. Rolul asociat unei persoane care practica meseria de calculator, era de a efectua calcule matematice complexe în funcție de necesitățile clientului. Odată cu trecerea timpului, știința a avut întâmpinat provocări la care persoanele calculator nu mai puteau ține pasul. Această lipsă a puterii de calcul și inventarea tranzistorului au concentrat mediul industrial și academic în dezvoltarea sistemelor pe care astăzi le folosim zilnic.

1.1.1 FPGA

În momentul de față, majoritatea dispozitivelor de calcul existente au la bază un procesor cu scop general care execută un program croit după necesitățile utilizatorului. În ciuda îmbunătățirilor constante și a flexibilității în dezvoltarea programelor acestor procesoare, arhitectura lor fixă nu se mulează întotdeauna suficient de mult pe problema pe care încearcă să o rezolve, rezultatul fiind nesatisfăcător din punctul de vedere a timpului de execuție și a amprentei energetice.

La jumătatea anilor 1980 a apărut primul circuit integrat comercial de tipul FPGA(Field-Programmable Gate Array)-referință. După cum le sugerează și numele, ele sunt niște circuite integrate care pot fi configurate de câte ori este nevoie de către utilizator. Avantajul acestor circuite este în principal natura lor paralelă și consumul redus de energie electrică. Datorită avansărilor tehnologice și prețului din ce în ce mai redus, ele au crescut în popularitate și au ajuns să fie folosite în multe domenii precum: *Calcul de înaltă performanță, Auto, Audio, Medical, etc.* - referință

1.1.2 Prolog

Prolog a fost inventat la începutul anilor 1970 de către Alain Colmerauer și colegii lui de la Universitatea din Marseille-ref. El este printre primele limbaje de uz general care au ca paradigmă de baza programarea logică. Prima implementare a acestui limbaj a fost sub forma unui interpretor scris în FORTRAN chiar de către grupul din care Colmerauer făcea parte. Chiar și sub această formă rudimentară realizările au fost mari deoarece s-a demonstrat viabilitatea unui limbaj de programare logic. După aproximativ un deceniu, David H. D. Warren a rafinat și abstractizat principiile de implementare a limbajului sub forma unei mașini abstracte care este cunoscută astăzi sub numele de WAM. Majoritatea implementărilor de success ale limbajului Prolog folosesc ca referință de bază mașina abstractă mai sus amintită datorită simplității, eficienței și robusteții care au trecut testul timpului. Datorită popularității WAM, au fost scrise multe lucrări științifice care au adus îmbunătățiri arhitecturale a mașinii inițiale cât și multe extinderi. Una dintre cele mai sensibile subiecte este paralelizarea eficientă a acestei arhitecturi în speranța de a obține rezultate practice mai bune.

Prolog este de obicei asociat cu domeniul inteligenței artificiale și a lingvisticii computaționale. Cele mai des întâlnite aplicații ale limbajului fac parte din sfera mediului academic—e.g, sisteme expert, demonstrarea automată a teoremelor, NLP, planificare automată, etc.

1.1.3 Nu m-am decis

În contextul actual, atât domeniul circuitelor reconfigurabile, cât și domeniul inteligenței artificiale se dezvoltă cu un ritm accelerat, fiind folosite tot mai des în produse comerciale—e.g., aproape toate produsele Google folosesc elemente de inteligență artificială (Translate, Photos, etc.)-ref, Microsoft exploatează circuitele reconfigurabile pentru a accelera motorul de căutare BING-ref. În principiu orice aplicație care are la bază inteligența artificială are nevoie de un mediu de antrenare sau executare de înaltă performanță pe care procesoarele obișnuite nu îl pot oferi. Încercarea mea urmărește să utilizeze natura paralelă și performanța crescută a circuitelor FPGA, pentru a soluționa problema puterii de calcul în contextul aplicațiilor bazate pe limbajul Prolog.

Capitolul 2

Obiectivele Proiectului

2.1 Scop

Scopul acestui proiect este de a realiza un sistem dedicat de execuție a programelor scrise în limbajul Prolog, urmând a fi folosit de către cercetători și ingineri care activează în domeniul inteligenței artificiale și a programării logice. Acest sistem va fi compus dintr-un procesor special optimizat pentru executarea unui set de instrucțiuni ales din limbajul Prolog, implementat pe un circuit reprogramabil de tipul FPGA, având la bază arhitectura WAM propusă de către David H. D. Warren. El va fi însoțit de o suită de programe compatibile cu sistemele de operare din familia UNIX, care vor servi ca interfață dintre calculatorul gazdă și noul procesor, FPWAM (FPGA PROLOG WAM).

2.2 Obiective generale

Până la sfârșitul dezvoltării acestui proiect, se urmăresc atingerea următoarelor obiective principale:

- Studiul în detaliu a mașinii propuse de către David H. D. Warren este imperios necesară deoarece ea va sta la baza procesorului FPWAM.
- Identificarea posibilelor instrucțiuni care pot fi optimizate.
- După studierea tuturor surselor teoretice, următorul pas va fi acela de a concepe o arhitectură fizică care să exploateze natura paralelă a circuitelor FPGA în implementarea algoritmilor prezenți în mașina WAM.
- Arhitectura proiectată să fie scalabilă, astfel încât să permită adăugarea cu ușurință a mai multor procesoare Prolog.
- Implementarea efectivă pe un circuit integrat de tip FPGA.

- Creșterea frecvenței de ceas prin metode de optimizare a circuitului.
- Din cauza faptului că proiectul de față nu își propune să construiască un compilator de Prolog, este necesară alegerea unui existent cu care FPWAM să fie compatibil.
- Dezvoltarea unui translator din instrucțiuni WAM în cod mașină FPWAM.
- Dezvoltarea unei aplicații de comunicare între calculatorul și FPWAM.
- Suita de programe dezvoltate să fie compatibile cu sistemele de operare din familia UNIX.

2.3 Obiective S.M.A.R.T.

Principiile S.M.A.R.T.

De cele mai multe ori scopurile și obiectivele sunt formulate fără o structură anume, având consecințe destul de grave precum imposibilitatea de urmărire progresul și lipsa de predictibilitate. Metodologia S.M.A.R.T. de formulare a obiectivelor și a scopurilor este folosită cel mai des în management-ul proiectelor fiind una dintre cele mai eficiente metode în acest sens. Pentru ca un obiectiv să îndeplinească criteriile S.M.A.R.T. el trebuie să îndeplinească următoarele principii:

- Specific: Cu cât obiectivul este mai specific, cu atât șansele sunt mai mari să-l obținem. El trebuie să răspundă la întrebări precum ce vrem să obținem exact, unde, cum sau când.
- Measurable (măsurabil): Ca un obiectiv să fie măsurabil el trebuie să fie împărțit în elementele măsurabile, rezultatul fiind apreciat după dovezile concrete.
- Attainable (realizabil): Este foarte important să punem în balanță efortul, timpul și alte costuri cu profitul și alte priorități pe care le avem.
- Relevant: Obiectivul trebuie să contribuie la atingerea scopului dorit.
- Time-bound (limitat de timp): Cu toții știm că termenele limită pun oamenii în mișcare, așa că obiectivele trebuie să fie constrânse de timp.

Obiectivele S.M.A.R.T. ale proiectului

Luând în considerare principiile enumerate mai sus, s-au formulat următoarele obiective S.M.A.R.T. ale proiectului:

- Realizarea unui studiu teoretic asupra implementărilor limbajului Prolog:
 - S: Studiarea și înțelegerea articolului scris de către David H. D. Warren în care introduce pentru prima oară arhitectura WAM. Studiarea a altor alternative.
 - M: Documentarea din cel puțin două surse diferite și schițarea unei sinteze a acestora.

- A: Prolog fiind un limbaj de programare popular, se găsesc multe surse din care se pot extrage informații relevante.
- R: Studiind soluțiile existente și elaborând o sinteză, putem să punem în balanță avantajele și dezavantajele lor, ducând în final la alegerea optimă.
- T: Studiul nu trebuie să depășească 3 luni de zile.
- Implementarea procesorului M0 pe FPGA:
 - S: În versiunea M0 se urmărește ca procesorul să poată unifica un fapt cu o întrebare Prolog predefinite în memoria de program.
 - M: Pentru ca acest obiectiv să fie îndeplinit următoarele elemente trebuie să fie implementate: arhitectura de bază formată din memorie și regiștrii, algoritmul de unificare a faptului cu întrebarea, un set minim de instrucțiuni WAM care să permită unificarea
 - A: Fiind la începutul proiectului, există timp pentru experimente până la definirea arhitecturii.
 - R: Atingerea acestui obiectiv este un pas foarte important deoarece demonstrează viabilitatea unei implementări hardware a limbajului.
 - T: Cu tot cu implementare și experimente, acest proces nu trebuie să dureze mai mult de 1 lună.
- Dezvoltarea suitei de programe pentru mediul de executare a FPWAM:
 - S: Suita de programe trebuie să permită compilarea și executarea de programe Prolog pe FPWAM. Modulul de compilare va utiliza GPLC (Gnu Prolog Compiler).
 - M: Obținerea unor executabile testate și compatibile cu sistemele de operare din familia UNIX.
 - A: Compilarea Prolog->WAM este efectuată de către GPLC reducând considerabil complexitatea acestui obiectiv.
 - R: Datorită acestor programe, interacțiunea dintre FPWAM și utilizator va fi posibilă într-un mod natural și eficient.
 - T: Funcționalitățile de baza trebuie să fie implementate în maxim 3 zile.
- Implementarea procesorului M3 pe FPGA:
 - S: În versiunea M3 procesorul va putea să execute orice program Prolog care respectă subsetul selectat de instrucțiuni.
 - A: Pentru ca acest obiectiv să fie îndeplinit următoarele funcționalități trebuie să fie implementate: mecanismul de apel a unui predicat, mecanismul de backtracking, 90% din instrucțiunile WAM.

- R: Acest obiectiv duce proiectul aproape de îndeplinirea scopului final. În acest stadiu se pot face comparații și teste amănunțite.
- T: Acest obiectiv trebuie să fie îndeplinit la cel puțin 15 zile înaintea de predarea proiectului.

Capitolul 3

Studiu Bibliografic

3.1 Sisteme de calcul reconfigurabile și FPGA

În ultimul deceniu, sistemele de calcul reconfigurabile au devenit foarte populare datorită capacității lor de a îmbina flexibilitatea unui produs software cu performanța ridicată a puterii de procesare a componentelor hardware. Acest concept a fost introdus pentru prima oară de către Gerlad Estrin într-un articol publicat în anii 1960-ref. El propunea un sistem format dintr-un procesor standard și o matrice de primitive hardware care putea fi configurată în funcție de necesitatea aplicației. Din cauza limitărilor tehnologice de la acea vreme, acest domeniu nu prea a fost în atenția cercetătorilor. După mai bine de două decenii producătorul de circuite integrate Xilinx inventează și lansează pentru prima oară circuitele field-programmable gate array (FPGA). Datorită comercializarea acestor circuite și performanțele ridicate ale acestora, domeniul sistemelor reconfigurabile a trecut printr-o perioadă de renaștere în care s-au publicat numeroase articole demonstrând viabilitatea acestor sisteme-ref.

3.1.1 Circuitele integrate FPGA

Un FPGA este un circuit integrat destinat configurării de către utilizator sau unui arhitect după procesul de fabricație. Configurația lor este specificată similar cu cea a circuitelor ASIC (application-specific integrated circuit) utilizând un limbaj de descriere hardware (HDL). Cea mai comună arhitectură pe care o adoptă aceste circuite conține o matrice de blocuri logice configurabile (CLB), celule de intrare/ieșire și canale de rutare. O imagine explicativă se găsește în figura 3.1.

Elementele CLB

Elementele CLB sunt blocurile fundamentale din care sunt formate FPGA-urile. Ele sunt alcătuite din mai multe celule logice (slice-uri). O versiune simplificată a unui slice se găsește în figura 3.2. De regulă el conține:

- LUT(look-up table) cu 4 intrări

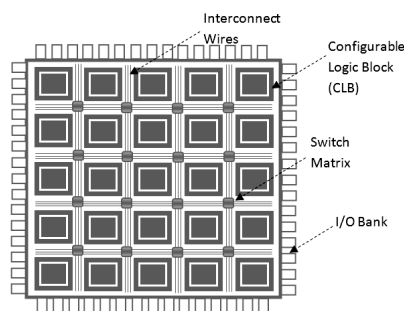


Figura 3.1: Arhitectura unui FPGA (<http://allaboutfpga.com/fpga-architecture/>)

- Sumator complet
- Bistabil de tip D
- Multiplexoare pentru selecția modului *normal sau aritmetic* în combinație cu modul de ieșire *sincron sau asincron*

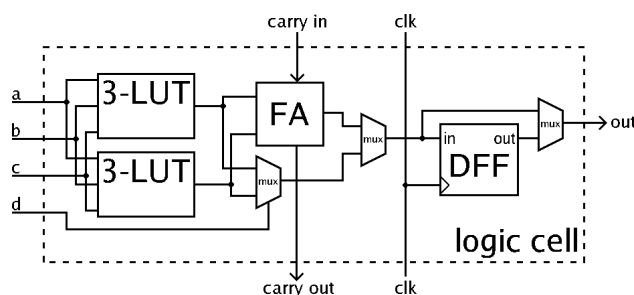


Figura 3.2: SLICE

Elemente de rutare

Elementele de rutare din FPGA sunt folosite pentru interconectarea blocurilor logice printr-o matrice de fire și comutatoare programabile. O schemă sumară se găsește în figura 3.3. În general rutarea între elementele programabile este nesegmentată, ceea ce înseamnă că fiecare segment rutat își găsește un capăt într-un bloc logic iar celălalt într-un comutator programabil. Pentru viteze ridicate, unele arhitecturi FPGA folosesc linii de rutare care trec prin mai multe elemente CLB.

3.2 Prolog și arhitectura WAM

După cum am amintit și în ?? limbajul Prolog a fost inventat la începutul anilor 1970 de către Alain Colmerauer și colegii lui de la Universitatea din Marseille-ref. El este printre primele

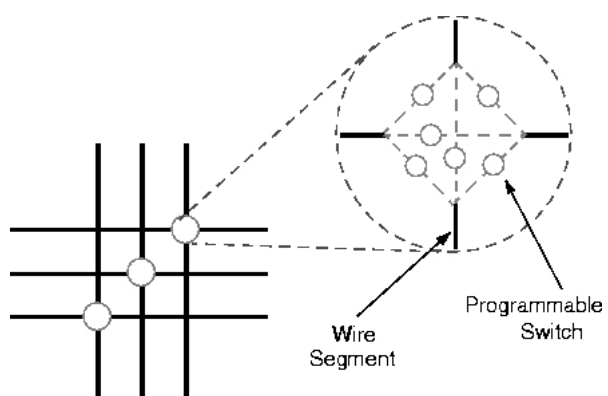


Figura 3.3: Matrice de rutare și comutator programabil

limbaje de uz general care au ca paradigmă de baza programarea logică.

Capitolul 4

Analiză și Fundamentare Teoretică

Împreună cu capitolul următor trebuie să reprezinte aproximativ 60% din total.

Scopul acestui capitol este de a explica principiile funcționale ale aplicației implementate. Aici se va descrie soluția propusă dintr-un punct de vedere teoretic - explicați și demonstrați proprietățile și valoarea teoretică:

- algoritm utilizat sau propus
- protocoale utilizate
- modele abstracte
- explicații/argumentări logice ale soluției alese
- structura logică și funcțională a aplicației.

NU SE FAC referiri la implementarea propriu-zisă.

NU SE PUN descrieri de tehnologii preluate cu copy-paste din alte surse sau lucruri care nu țin strict de proiectul propriu-zis (materiale de umplură).

4.1 Titlu

4.2 Alt titlu

Capitolul 5

Proiectare de Detaliu și Implementare

Împreună cu capitolul precedent reprezintă aproximativ 60% din total.

Scopul acestui capitol este de a documenta aplicația dezvoltată în așa fel încât dezvoltarea și întreținerea ulterioară să fie posibile. Cititorul trebuie să identifice funcțiile principale ale aplicației din ceea ce este scris aici. Capitolul ar trebui să conțină (nu se rezumă neapărat la):

- schema generală a aplicației
- descrierea fiecărei componente implementate, la nivel de modul
- diagrame de clase, clase importante și metode ale claselor importante.

Capitolul 6

Testare și Validare

Aproximativ 5% din total

6.1 Titlu

6.2 Alt titlu

Capitolul 7

Manual de Instalare și Utilizare

În secțiunea de Instalare trebuie să detaliați resursele software și hardware necesare pentru instalarea și rularea aplicației, precum și o descriere pas cu pas a procesului de instalare. Instalarea aplicației trebuie să fie posibilă pe baza a ceea ce se scrie aici.

În acest capitol trebuie să descrieți cum se utilizează aplicația din punct de vedere al utilizatorului, fără a menționa aspecte tehnice interne. Folosiți capturi ale ecranului și explicații pas cu pas ale interacțiunii. Folosind acest manual, o persoană ar trebui să poată utiliza produsul vostru.

7.1 Titlu

7.2 Alt titlu

Capitolul 8

Concluzii

Cca. 5% din total. Capitolul ar trebui sa conțină (nu se rezumă neapărat la):

- un rezumat al contribuțiilor voastre
- analiză critică a rezultatelor obținute
- descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

8.1 Titlu

8.2 Alt titlu

Bibliografie

- [1] W. Strunk, Jr. and E. B. White, *The Elements of Style*, 3rd ed. Macmillan, 1979.
- [2] E. Bellucci, A. Lodder, and J. Zeleznikow, “Integrating artificial intelligence, argumentation and game theory to develop an online dispute resolution environment.’ in *16th International Conference on Tools with Artificial Intelligence*, 2004, pp. 749–754.
- [3] G. Antoniou, T. Skylogiannis, A. Bikakis, M. Doerr, and N. Bassiliades, “Dr-brokering: A semantic brokering system.’ *Knowledge-Based Systems*, vol. 20, no. 1, pp. 61–72, 2007.
- [4] S. J. Russell, P. Norvig, J. F. Canny, J. M. Malik, and D. D. Edwards, *Artificial intelligence: a modern approach*. Prentice hall Englewood Cliffs, 1995, vol. 2.
- [5] “Ajax tutorial.’ [Online]. Available: <http://www.tutorialspoint.com/ajax/>.

Anexa A

Secțiuni relevante din cod

```
/** Maps are easy to use in Scala. */
object Maps {
  val colors = Map("red" -> 0xFF0000,
                   "turquoise" -> 0x00FFFF,
                   "black" -> 0x000000,
                   "orange" -> 0xFF8040,
                   "brown" -> 0x804000)

  def main(args: Array[String]) {
    for (name <- args) println(
      colors.get(name) match {
        case Some(code) =>
          name + " has code: " + code
        case None =>
          "Unknown color: " + name
      }
    )
  }
}
```

Anexa B

**Alte informații relevante (demonstrații
etc.)**

Anexa C

Lucrări publicate (dacă există)